Code -

```
#include <iostream>
#include <vector>
#include <iomanip>

using namespace std;

const int INF = 999;

// ------------------------------------------------------------
// Structure for routing table entry
// ------------------------------------------------------------
struct Route
{
    int cost;
    int nextHop;
};


// ------------------------------------------------------------
// Convert router number to router name
// 0 -> A
// 1 -> B
// 2 -> C
// 3 -> D
// ------------------------------------------------------------
char routerName(int router)
{
    return 'A' + router;
}


// ------------------------------------------------------------
// Convert router name to router number
// A -> 0
// B -> 1
// C -> 2
// D -> 3
// ------------------------------------------------------------
int routerNumber(char router)
{
    return router - 'A';
}


// ------------------------------------------------------------
// Print routing tables
// ------------------------------------------------------------
void printRoutingTables(const vector<vector<Route>>& table)
{
    int n = table.size();

    for (int router = 0; router < n; router++)
```

```
    {
        cout << "\n-------------------------------------------\n";
        cout << "Routing Table of Router "
             << routerName(router) << "\n";
        cout << "-------------------------------------------\n";

        cout << left
             << setw(15) << "Destination"
             << setw(15) << "Cost"
             << setw(15) << "Next Hop"
             << "\n";

        cout << "-------------------------------------------\n";

        for (int destination = 0;
             destination < n;
             destination++)
        {
            cout << left
                 << setw(15)
                 << routerName(destination);

            if (table[router][destination].cost >= INF)
            {
                cout << setw(15) << "INF";
            }
            else
            {
                cout << setw(15)
                     << table[router][destination].cost;
            }

            if (table[router][destination].nextHop == -1)
            {
                cout << setw(15) << "-";
            }
            else
            {
                cout << setw(15)
                     << routerName(
                            table[router][destination].nextHop
                        );
            }

            cout << "\n";
        }
    }
}


// ------------------------------------------------------
// Print shortest path from source to destination
// ------------------------------------------------------
```

```
void printShortestPath(
    const vector<vector<Route>>& routingTable,
    int source,
    int destination)
{
    int n = routingTable.size();

    cout << "\n============================================================\n";
    cout << "                SHORTEST PATH RESULT\n";
    cout << "============================================================\n";

    cout << "Source           : "
         << routerName(source) << "\n";

    cout << "Destination      : "
         << routerName(destination) << "\n";

    // Check whether destination is reachable
    if (routingTable[source][destination].cost >= INF)
    {
        cout << "Shortest Distance: INF\n";
        cout << "Shortest Path    : No path exists\n";
        return;
    }

    cout << "Shortest Distance: "
         << routingTable[source][destination].cost
         << "\n";

    cout << "Shortest Path    : ";

    // Start from source
    int current = source;

    cout << routerName(current);

    // Follow next hops until destination
    int count = 0;

    while (current != destination)
    {
        int next =
            routingTable[current][destination].nextHop;

        // Safety check to avoid infinite loop
        if (next == -1 || count >= n)
        {
            cout << "\nError: Unable to construct path.\n";
            return;
        }

        cout << " -> "
             << routerName(next);
```

```
        current = next;
        count++;
    }

    cout << "\n";
}


// ---------------------------------------------------------
// Main function
// ---------------------------------------------------------
int main()
{
    const int n = 4;

    cout << "============================================================\n";
    cout << "            DISTANCE VECTOR ROUTING\n";
    cout << "============================================================\n";

    cout << "\nRouters: A, B, C, D\n";


    // ---------------------------------------------------------
    // Cost matrix
    //
    // A --2-- B
    // |       |
    // 1       3
    // |       |
    // D --11- C
    //
    // B --7-- D
    //
    // INF means no direct connection
    // ---------------------------------------------------------

    vector<vector<int>> cost =
    {
        {0,   2,   INF, 1},
        {2,   0,   3,   7},
        {INF, 3,   0,   11},
        {1,   7,   11, 0}
    };


    // ---------------------------------------------------------
    // Display cost matrix
    // ---------------------------------------------------------

    cout << "\nCost Matrix:\n\n";

    cout << setw(10) << " ";
```

```
for (int i = 0; i < n; i++)
{
    cout << setw(8)
         << routerName(i);
}

cout << "\n";

for (int i = 0; i < n; i++)
{
    cout << setw(8)
         << routerName(i);

    for (int j = 0; j < n; j++)
    {
        if (cost[i][j] >= INF)
            cout << setw(8) << "INF";
        else
            cout << setw(8) << cost[i][j];
    }

    cout << "\n";
}


// ------------------------------------------------------
// Initialize routing tables
// ------------------------------------------------------

vector<vector<Route>> routingTable(
    n,
    vector<Route>(n)
);


for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++)
    {
        // Router to itself
        if (i == j)
        {
            routingTable[i][j].cost = 0;
            routingTable[i][j].nextHop = i;
        }

        // Direct connection
        else if (cost[i][j] < INF)
        {
            routingTable[i][j].cost = cost[i][j];
            routingTable[i][j].nextHop = j;
        }
```

```
        // No direct connection
        else
        {
            routingTable[i][j].cost = INF;
            routingTable[i][j].nextHop = -1;
        }
    }
}


// ---------------------------------------------------
// Initial routing tables
// ---------------------------------------------------

cout << "\n\n============================================================\n";
cout << "                INITIAL ROUTING TABLES\n";
cout << "============================================================\n";

printRoutingTables(routingTable);


// ---------------------------------------------------
// Distance Vector Algorithm
// ---------------------------------------------------

int iteration = 0;

while (true)
{
    iteration++;

    bool updated = false;

    // Use old routing table for this iteration
    vector<vector<Route>> newTable =
        routingTable;


    // ---------------------------------------------------
    // For every router
    // ---------------------------------------------------

    for (int i = 0; i < n; i++)
    {
        // -----------------------------------------------
        // Consider every neighbor
        // -----------------------------------------------

        for (int neighbor = 0;
             neighbor < n;
             neighbor++)
        {
```

```
    // No direct connection
    if (i == neighbor ||
        cost[i][neighbor] >= INF)
    {
        continue;
    }


    // ------------------------------------------------
    // Consider every destination
    // ------------------------------------------------

    for (int destination = 0;
         destination < n;
         destination++)
    {
        // Neighbor cannot reach destination
        if (routingTable[neighbor][destination].cost
            >= INF)
        {
            continue;
        }


        // ------------------------------------------------
        // Bellman-Ford equation
        //
        // D(i,j) =
        // min [ C(i,k) + D(k,j) ]
        // ------------------------------------------------

        int newCost =
            cost[i][neighbor] +
            routingTable[neighbor][destination].cost;


        // ------------------------------------------------
        // Found shorter path
        // ------------------------------------------------

        if (newCost <
            newTable[i][destination].cost)
        {
            newTable[i][destination].cost =
                newCost;

            newTable[i][destination].nextHop =
                neighbor;

            updated = true;
        }
    }
}
```

```
        }


        // Update routing table
        routingTable = newTable;


        // --------------------------------------------------
        // Display current iteration
        // --------------------------------------------------

        cout <<
"\n\n============================================================\n";
        cout << "                  ITERATION "
             << iteration << "\n";
        cout << "============================================================\n";

        printRoutingTables(routingTable);


        // --------------------------------------------------
        // Stop when no changes occur
        // --------------------------------------------------

        if (!updated)
        {
            break;
        }


        // Safety limit
        if (iteration > n)
        {
            break;
        }
    }


    // ------------------------------------------------------
    // Final routing tables
    // ------------------------------------------------------

    cout << "\n\n============================================================\n";
    cout << "             FINAL CONVERGED ROUTING TABLES\n";
    cout << "============================================================\n";

    printRoutingTables(routingTable);


    cout << "\nAlgorithm converged after "
         << iteration
         << " iteration(s).\n";
```

```
    // ------------------------------------------------
    // Take source and destination from user
    // ------------------------------------------------

    char sourceRouter;
    char destinationRouter;

    cout << "\n============================================================\n";
    cout << "                FIND SHORTEST PATH\n";
    cout << "============================================================\n";

    cout << "\nEnter Source Router (A-D): ";
    cin >> sourceRouter;

    cout << "Enter Destination Router (A-D): ";
    cin >> destinationRouter;


    // Convert characters to indexes
    int source = routerNumber(sourceRouter);
    int destination = routerNumber(destinationRouter);


    // ------------------------------------------------
    // Validate input
    // ------------------------------------------------

    if (source < 0 || source >= n ||
        destination < 0 || destination >= n)
    {
        cout << "\nInvalid router entered.\n";
        return 0;
    }


    // ------------------------------------------------
    // Display shortest distance and path
    // ------------------------------------------------

    printShortestPath(
        routingTable,
        source,
        destination
    );


    return 0;
}
```

Output -

```
============================================================
            DISTANCE VECTOR ROUTING
```

==========

Routers: A, B, C, D

Cost Matrix:

	A	B	C	D
A	0	2	INF	1
B	2	0	3	7
C	INF	3	0	11
D	1	7	11	0

==========
INITIAL ROUTING TABLES
==========

Routing Table of Router A

Destination Cost Next Hop

A 0 A
B 2 B
C INF -
D 1 D

Routing Table of Router B

Destination Cost Next Hop

A 2 A
B 0 B
C 3 C
D 7 D

Routing Table of Router C

Destination Cost Next Hop

A INF -
B 3 B
C 0 C
D 11 D

Routing Table of Router D

Destination Cost Next Hop

A 1 A

```
B           7           B
C           11           C
D           0           D


============================================================
                ITERATION 1
============================================================

--------------------------------------------
Routing Table of Router A
--------------------------------------------
Destination    Cost          Next Hop
--------------------------------------------
A           0           A
B           2           B
C           5           B
D           1           D

--------------------------------------------
Routing Table of Router B
--------------------------------------------
Destination    Cost          Next Hop
--------------------------------------------
A           2           A
B           0           B
C           3           C
D           3           A

--------------------------------------------
Routing Table of Router C
--------------------------------------------
Destination    Cost          Next Hop
--------------------------------------------
A           5           B
B           3           B
C           0           C
D           10           B

--------------------------------------------
Routing Table of Router D
--------------------------------------------
Destination    Cost          Next Hop
--------------------------------------------
A           1           A
B           3           A
C           10           B
D           0           D


============================================================
                ITERATION 2
============================================================
```

Routing Table of Router A

Destination	Cost	Next Hop
A	0	A
B	2	B
C	5	B
D	1	D

Routing Table of Router B

Destination	Cost	Next Hop
A	2	A
B	0	B
C	3	C
D	3	A

Routing Table of Router C

Destination	Cost	Next Hop
A	5	B
B	3	B
C	0	C
D	6	B

Routing Table of Router D

Destination	Cost	Next Hop
A	1	A
B	3	A
C	6	A
D	0	D

==========
ITERATION 3
==========

Routing Table of Router A

Destination	Cost	Next Hop
A	0	A
B	2	B

C 5 B
D 1 D

Routing Table of Router B

Destination Cost Next Hop

A 2 A
B 0 B
C 3 C
D 3 A

Routing Table of Router C

Destination Cost Next Hop

A 5 B
B 3 B
C 0 C
D 6 B

Routing Table of Router D

Destination Cost Next Hop

A 1 A
B 3 A
C 6 A
D 0 D

==========
FINAL CONVERGED ROUTING TABLES
==========

Routing Table of Router A

Destination Cost Next Hop

A 0 A
B 2 B
C 5 B
D 1 D

Routing Table of Router B

Destination Cost Next Hop

```
A           2           A
B           0           B
C           3           C
D           3           A

------------------------------------------
Routing Table of Router C
------------------------------------------
Destination    Cost          Next Hop
------------------------------------------
A           5           B
B           3           B
C           0           C
D           6           B

------------------------------------------
Routing Table of Router D
------------------------------------------
Destination    Cost          Next Hop
------------------------------------------
A           1           A
B           3           A
C           6           A
D           0           D

Algorithm converged after 3 iteration(s).

============================================================
               FIND SHORTEST PATH
============================================================

Enter Source Router (A-D): A
Enter Destination Router (A-D): C

============================================================
               SHORTEST PATH RESULT
============================================================
Source          : A
Destination     : C
Shortest Distance: 5
Shortest Path    : A -> B -> C
```